Manual — Kiro + Ollama + Qwen2.5-Coder 14B + MCP

Kiro como IA principal + Qwen local para segunda opinião, revisão, debugging, testes e performance

1. Arquitetura

O Kiro permanece como agente principal. O Qwen2.5-Coder 14B roda localmente no Ollama e é chamado pelo Kiro através de MCP quando uma segunda opinião é útil.

```
Você
  ↓
Kiro AI — agente principal
  ↓
MCP: ollama-local
  ↓
Python server.py
  ↓ HTTP localhost:11434
Ollama
  ↓
Qwen2.5-Coder 14B
  ↓
RTX 2060 12 GB
```

2. Modelo escolhido

Foi testado qwen3-coder:latest (~19 GB), que dividia a execução entre CPU e GPU. Para a RTX 2060 12 GB, usamos qwen2.5-coder:14b (~9,5 GB), observado em 100% GPU.

```
qwen2.5-coder:14b
Tamanho: ~9.5 GB
GPU: 100%
Contexto: 4096
Keep-alive: 15 minutos
```

3. Instalação do Ollama

Instale o Ollama para Windows pelo site oficial: https://ollama.com/

```
ollama --version
ollama list
```

A API local utilizada pelo MCP fica em http://localhost:11434.

4. Baixar e testar o Qwen

```
ollama pull qwen2.5-coder:14b
ollama run qwen2.5-coder:14b

Explique o que é uma race condition em C++.
```

Para sair do modo interativo, use /bye.

5. Verificar GPU

```
ollama ps
```

Esperado neste computador:

```
qwen2.5-coder:14b    9.5 GB    100% GPU    4096
```

6. Python e MCP SDK

```
py --version
py -m pip --version

py -m pip install "mcp[cli]"

py -c "import mcp; print('MCP OK')
```

O Node.js não é necessário para esta arquitetura.

7. Criar o servidor MCP

```
C:\AI\kiro-ollama-mcp\
■■■ server.py
```

7.1 server.py

```
import json
import urllib.request
from mcp.server.mcpserver import MCPServer

OLLAMA_URL = "http://localhost:11434"
DEFAULT_MODEL = "qwen2.5-coder:14b"
KEEP_ALIVE = "15m"

mcp = MCPServer("ollama-local")

def ollama_generate(model: str, prompt: str) -> str:
    payload = json.dumps({
        "model": model,
        "prompt": prompt,
        "stream": False,
        "keep_alive": KEEP_ALIVE
    }).encode("utf-8")

    request = urllib.request.Request(
        f"{OLLAMA_URL}/api/generate",
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST"
    )

    with urllib.request.urlopen(request, timeout=300) as response:
        data = json.loads(response.read().decode("utf-8"))

    return data.get("response", "")

@mcp.tool()
def ask_local_ai(prompt: str) -> str:
    enhanced_prompt = f"""
You are a senior software engineer performing a second-opinion
analysis for another AI coding agent.

Priorities:
- Find real and relevant problems.
- Do not invent problems.
- Do not recommend premature optimization.
- Distinguish real bugs from style preferences.
- Consider the existing context.

Analyze:
1. Memory leaks and memory safety
2. Undefined behavior
3. Race conditions
4. Thread safety and concurrency
5. Performance
6. Error handling
7. Security
8. Maintainability
9. Code smells
10. Refactoring opportunities

For each relevant problem provide:
- Severity: Critical / High / Medium / Low / Informational
- Location
- Problem
- Why it matters
- Recommended solution

If a category has no relevant problems, say so.
Do not modify files.
Do not rewrite the entire code unless requested.

USER REQUEST:
{prompt}
"""
    return ollama_generate(DEFAULT_MODEL, enhanced_prompt)

@mcp.tool()
```

```
def local_ai_status() -> str:
    try:
        request = urllib.request.Request(
            f"{OLLAMA_URL}/api/tags", method="GET"
        )
        with urllib.request.urlopen(request, timeout=5) as response:
            data = json.loads(response.read().decode("utf-8"))

        models = [m.get("name") for m in data.get("models", [])]

        return (
            "Ollama is running.\n"
            f"Default model: {DEFAULT_MODEL}\n"
            f"Keep alive: {KEEP_ALIVE}\n"
            f"Available models: {', '.join(models)}"
        )
    except Exception as error:
        return f"Ollama is not available: {error}"

if __name__ == "__main__":
    mcp.run()
```

8. Configurar MCP no Kiro

Arquivo global utilizado:

```
C:\Users\danni\.kiro\settings\mcp.json
```

```
{
  "mcpServers": {
    "ollama-local": {
      "command": "C:\\Python314\\python.exe",
      "args": [
        "C:\\AI\\kiro-ollama-mcp\\server.py"
      ],
      "disabled": false
    }
  }
}
```

Em outra máquina, ajuste o caminho do Python e do server.py. Se necessário, descubra o Python com: py -c "import sys; print(sys.executable)".

9. Reiniciar e testar

• Salvar mcp.json.

• Fechar o Kiro.

• Abrir novamente.

• Abrir o projeto.

```
Use a ferramenta ask_local_ai para analisar este código.
Não altere nenhum arquivo.
```

Depois do teste, use ollama ps para confirmar que o Qwen está carregado e em GPU.

10. Keep-alive

O servidor envia keep_alive=15m para a API do Ollama. Assim o modelo permanece carregado por aproximadamente 15 minutos após uma chamada.

```
KEEP_ALIVE = "15m"
...
"keep_alive": KEEP_ALIVE
```

11. Steering do Kiro

```
.kiro/
■■■ steering/
    ■■■ debug.md
    ■■■ local-ai.md
    ■■■ perf.md
    ■■■ review.md
    ■■■ tests.md
```

Os arquivos de comando usam frontmatter:

```
---
inclusion: manual
---
```

12. Comandos

Comando	Função
/local	Segunda opinião local genérica
/review	Revisão de código: memória, UB, concorrência, performance, segurança etc.
/debug	Investigação de bugs e hipóteses
/tests	Estratégia, casos e geração de testes
/perf	Gargalos e análise de performance

13. Fluxo diário

```
Desenvolvimento → Kiro

/review → Kiro → Qwen → Kiro → resultado
/debug  → Kiro → Qwen → Kiro → diagnóstico
/tests  → Kiro → Qwen → Kiro → estratégia
/perf   → Kiro → Qwen → Kiro → análise
/local  → Kiro → Qwen → Kiro → segunda opinião
```

A ideia é não precisar pensar no Ollama durante o desenvolvimento normal.

14. Estrutura final

```
C:\AI\kiro-ollama-mcp\
■■■ server.py

C:\Users\danni\.kiro\
■■■ settings\
    ■■■ mcp.json

projeto_teste\
■■■ .kiro\
    ■■■ steering\
        ■■■ debug.md
        ■■■ local-ai.md
        ■■■ perf.md
        ■■■ review.md
        ■■■ tests.md
```

15. O que não precisamos

• Node.js

• Docker

• Open WebUI

• LangChain

• LlamaIndex

• Agentes adicionais

• Rodar ollama run manualmente para cada uso do Kiro.

16. Diagnóstico rápido

Ollama

```
ollama list
ollama ps
```

Modelo

```
ollama pull qwen2.5-coder:14b
```

Kiro não encontra MCP

• Verifique %USERPROFILE%\.kiro\settings\mcp.json.

• Confira os caminhos do Python e server.py.

• Feche e abra o Kiro novamente.

Steering não aparece

• Confirme .kiro/steering no projeto.

• Confirme inclusion: manual.

• Feche e abra o Kiro novamente.

CPU/GPU

Se o modelo não estiver em 100% GPU, confira o tamanho do modelo e a VRAM disponível. Neste setup, o Qwen 14B ficou em 100% GPU.

17. Segurança

Mantenha a API do Ollama em localhost. Não há necessidade de expor a porta 11434 à internet. Trate MCPs locais como software confiável, pois processos MCP podem acessar recursos disponíveis para o processo.

18. Checklist de instalação do zero

• ■ Instalar Kiro

• ■ Instalar Ollama

• ■ ollama --version

• ■ ollama pull qwen2.5-coder:14b

• ■ ollama run qwen2.5-coder:14b

• ■ ollama ps → confirmar GPU

• ■ Instalar/verificar Python

• ■ pip install "mcp[cli]"

• ■ Criar server.py

• ■ Configurar mcp.json

• ■ Reiniciar Kiro

• ■ Testar ask_local_ai

• ■ Criar .kiro/steering

• ■ Configurar /local /review /debug /tests /perf

• ■ Rodar /review e verificar a segunda opinião do Qwen

19. Estado final

```
IA principal:      Kiro AI
IA local:           Qwen2.5-Coder 14B
Runtime:            Ollama
Integração:         MCP
Servidor:           Python
Keep-alive:         15 minutos
GPU:                RTX 2060 12 GB
Modelo:             ~9.5 GB / 100% GPU
Contexto:           4096
```

```
Comandos:
  /local
  /review
  /debug
  /tests
  /perf
```

Referências oficiais: Ollama — https://ollama.com/ | Kiro MCP — https://kiro.dev/docs/mcp/configuration/ | Kiro Steering — https://kiro.dev/docs/steering/ | MCP Python SDK — https://py.sdk.modelcontextprotocol.io/